DevOps: CI/CD Pipeline

Assignment Submission

Student Name: Rohan Mangate_12233

Assignment Title: DevOps: CI/CD Pipeline

Repository URL: https://github.com/RohanMangate/python-flask-sample-app

1. Introduction

This assignment demonstrates the implementation of Continuous Integration and Continuous Deployment (CI/CD) pipelines using Jenkins and GitHub Actions for a Python Flask application. The pipelines automate dependency installation, unit testing, staging deployment, and production deployment using release-based triggers.

2. Jenkins CI/CD Pipeline for Flask Application

2.1 Objective

To set up a Jenkins pipeline that automates the build, testing, and deployment of a Python Flask web application.

2.2 Jenkins Setup

Jenkins was installed on an AWS EC2 virtual machine running Ubuntu. Java and Python were configured on the server, along with required Python libraries. Jenkins pipelines are managed using a Jenkinsfile stored in the GitHub repository.

EC2 instance

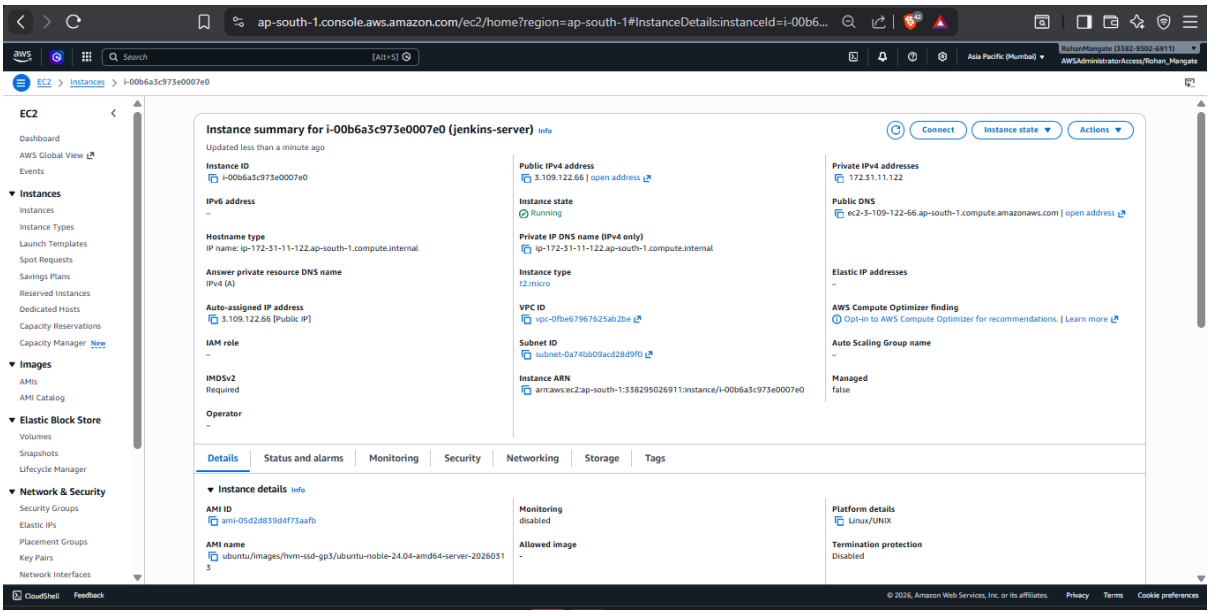
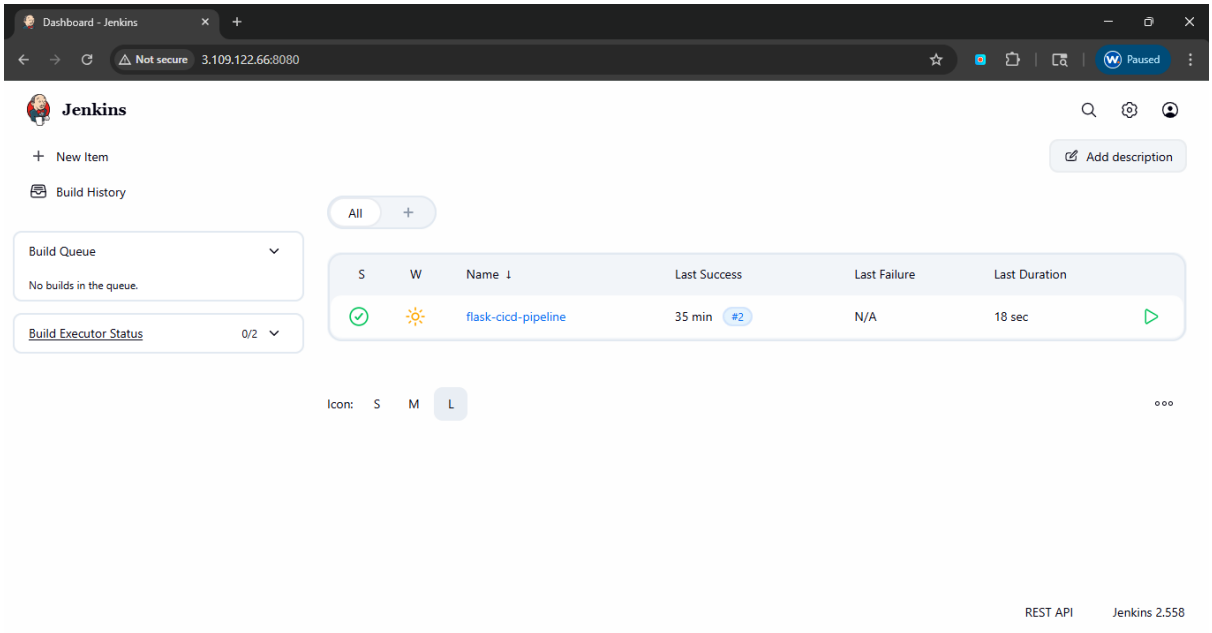
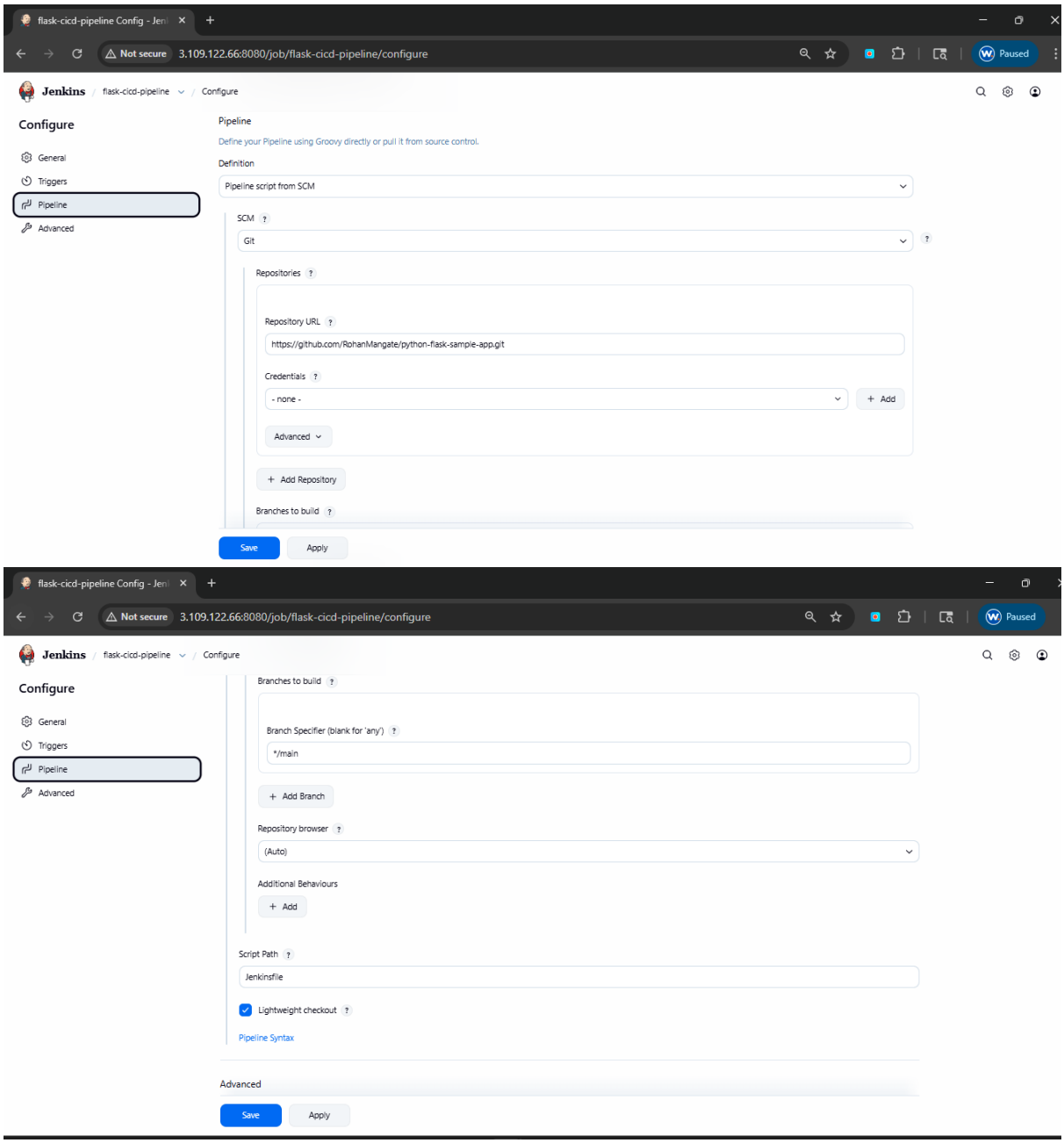


Figure 1: Jenkins Dashboard



This screenshot shows the Jenkins dashboard running on an AWS EC2 instance. The pipeline job flask-cicd-pipeline is visible with a successful build status, confirming that Jenkins has been installed and configured correctly.

Figure 2: Jenkins Pipeline Configuration Using Git SCM



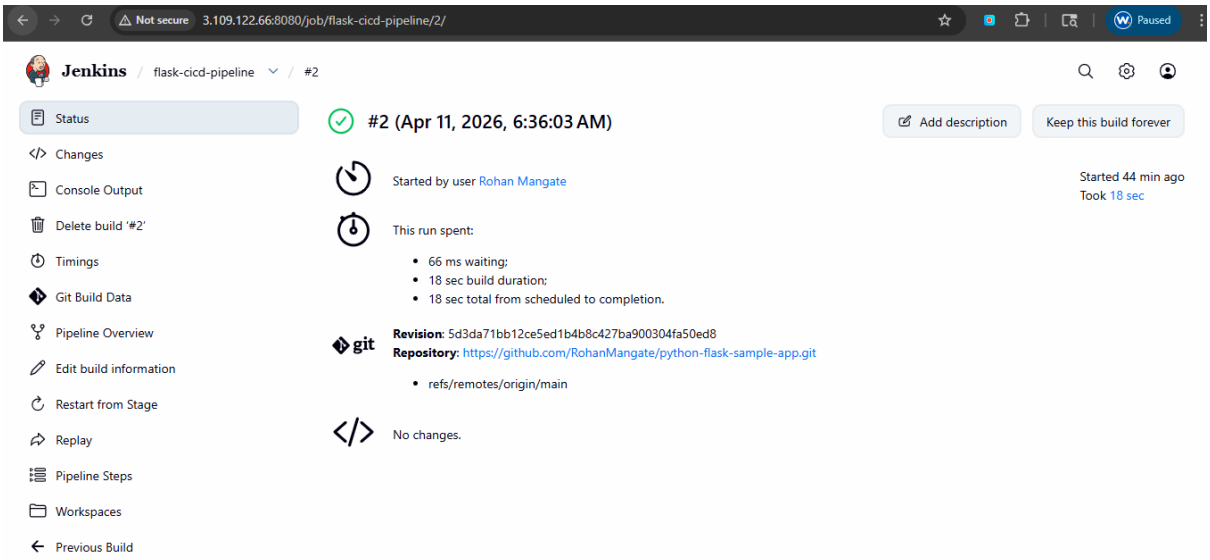
This screenshot shows the Jenkins pipeline configuration where the job is defined using "Pipeline script from SCM". Jenkins pulls source code from a GitHub repository and executes the Jenkinsfile from the main branch.

2.3 Jenkins Pipeline Stages

The Jenkins pipeline is defined using a declarative Jenkinsfile with the following stages:

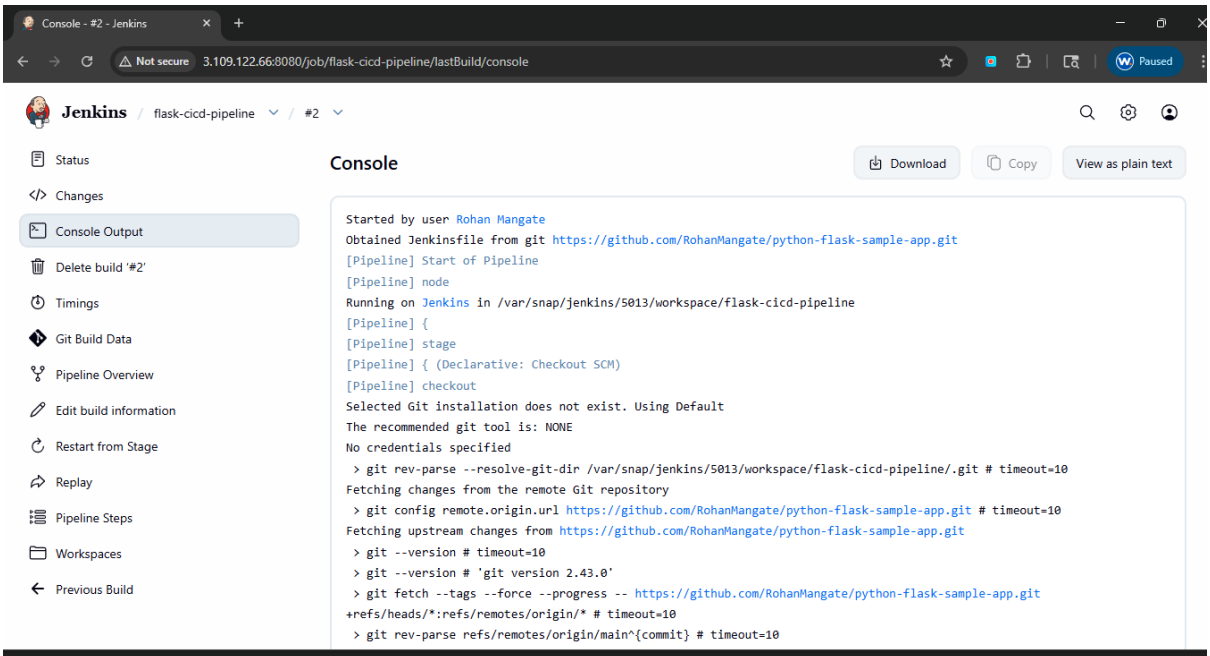
Stage	Description
Build	Installs application dependencies using pip.
Test	Runs unit tests using pytest.
Deploy	Deploys the Flask application to a staging environment after tests pass.

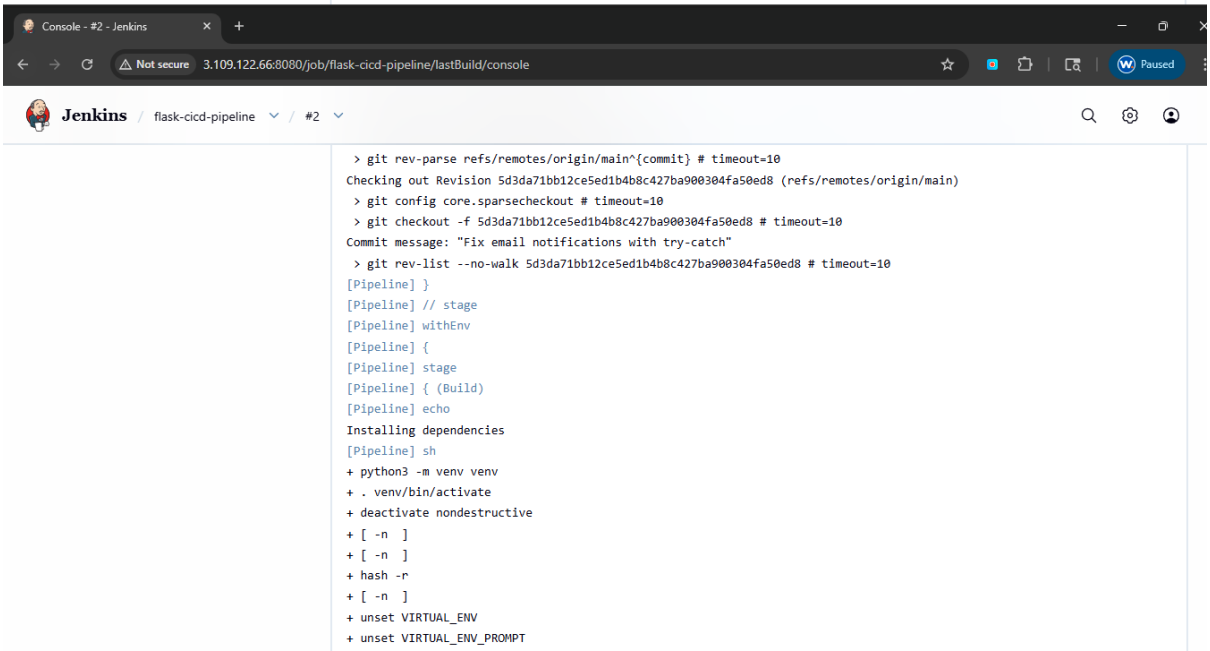
Figure 3: Jenkins Pipeline Successful Build Execution



This screenshot shows a successful Jenkins pipeline run indicating that all configured stages executed without errors.

Figure 4: Jenkins Console Output Showing Build, Test, and Deploy Stages





Console - #2 - Jenkins

Not secure 3.109.122.66:8080/job/flask-cicd-pipeline/lastBuild/console

Jenkins / flask-cicd-pipeline / #2

```
+ unset VIRTUAL_ENV_PROMPT
+ [ ! nondestructive = nondestructive ]
+ [  = cygwin ]
+ [  = msys ]
+ export VIRTUAL_ENV=/var/snap/jenkins/5013/workspace/flask-cicd-pipeline/venv
+ _OLD_VIRTUAL_PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
+ PATH=/var/snap/jenkins/5013/workspace/flask-cicd-
pipeline/venv/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
+ export PATH
+ [ -n  ]
+ [ -z  ]
+ _OLD_VIRTUAL_PS1=#
+ PS1=(venv) #
+ export PS1
+ VIRTUAL_ENV_PROMPT=(venv)
+ export VIRTUAL_ENV_PROMPT
+ hash -r
+ pip install --upgrade pip
Requirement already satisfied: pip in ./venv/lib/python3.12/site-packages (26.0.1)
+ pip install -r requirements.txt
Requirement already satisfied: flask==3.0.0 in ./venv/lib/python3.12/site-packages (from -r requirements.txt (line 1))
(3.0.0)
Requirement already satisfied: pytest==7.4.3 in ./venv/lib/python3.12/site-packages (from -r requirements.txt (line 2))
(7.4.3)
```

Console - #2 - Jenkins

Not secure 3.109.122.66:8080/job/flask-cicd-pipeline/lastBuild/console

Jenkins / flask-cicd-pipeline / #2

```
(7.4.3)
Requirement already satisfied: Werkzeug>=3.0.0 in ./venv/lib/python3.12/site-packages (from flask==3.0.0->-r
requirements.txt (line 1)) (3.1.8)
Requirement already satisfied: Jinja2>=3.1.2 in ./venv/lib/python3.12/site-packages (from flask==3.0.0->-r
requirements.txt (line 1)) (3.1.6)
Requirement already satisfied: itsdangerous>=2.1.2 in ./venv/lib/python3.12/site-packages (from flask==3.0.0->-r
requirements.txt (line 1)) (2.2.0)
Requirement already satisfied: click>=8.1.3 in ./venv/lib/python3.12/site-packages (from flask==3.0.0->-r
requirements.txt (line 1)) (8.3.2)
Requirement already satisfied: blinker>=1.6.2 in ./venv/lib/python3.12/site-packages (from flask==3.0.0->-r
requirements.txt (line 1)) (1.9.0)
Requirement already satisfied: iniconfig in ./venv/lib/python3.12/site-packages (from pytest==7.4.3->-r
requirements.txt (line 2)) (2.3.0)
Requirement already satisfied: packaging in ./venv/lib/python3.12/site-packages (from pytest==7.4.3->-r
requirements.txt (line 2)) (26.0)
Requirement already satisfied: pluggy<2.0,>=0.12 in ./venv/lib/python3.12/site-packages (from pytest==7.4.3->-r
requirements.txt (line 2)) (1.6.0)
Requirement already satisfied: MarkupSafe>=2.0 in ./venv/lib/python3.12/site-packages (from Jinja2>=3.1.2-
>flask==3.0.0->-r requirements.txt (line 1)) (3.0.3)
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
```

Console - #2 - Jenkins

Not secure 3.109.122.66:8080/job/flask-cicd-pipeline/lastBuild/console

Jenkins / flask-cicd-pipeline / #2

```
Running unit tests
[Pipeline] sh
+ . venv/bin/activate
+ deactivate nondestructive
+ [ -n  ]
+ [ -n  ]
+ hash -r
+ [ -n  ]
+ unset VIRTUAL_ENV
+ unset VIRTUAL_ENV_PROMPT
+ [ ! nondestructive = nondestructive ]
+ [  = cygwin ]
+ [  = msys ]
+ export VIRTUAL_ENV=/var/snap/jenkins/5013/workspace/flask-cicd-pipeline/venv
+ _OLD_VIRTUAL_PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
+ PATH=/var/snap/jenkins/5013/workspace/flask-cicd-
pipeline/venv/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
+ export PATH
+ [ -n  ]
+ [ -z  ]
+ _OLD_VIRTUAL_PS1=#
+ PS1=(venv) #
+ export PS1
+ VIRTUAL_ENV_PROMPT=(venv)
```

Console - #2 - Jenkins

Not secure 3.109.122.66:8080/job/flask-cicd-pipeline/lastBuild/console

Jenkins / flask-cicd-pipeline / #2

```
+ VIRTUAL_ENV_PROMPT=(venv)
+ export VIRTUAL_ENV_PROMPT
+ hash -r
+ pytest test_app.py -v
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-7.4.3, pluggy-1.6.0 -- /var/snap/jenkins/5013/workspace/flask-cicd-
pipeline/venv/bin/python3
cachedir: .pytest_cache
rootdir: /var/snap/jenkins/5013/workspace/flask-cicd-pipeline
collecting ... collected 3 items

test_app.py::test_home PASSED                                            [ 33%]
test_app.py::test_health PASSED                                          [ 66%]
test_app.py::test_about PASSED                                           [100%]

============================== 3 passed in 0.33s ===============================
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploying application to staging
[Pipeline] sh
+ . venv/bin/activate
```

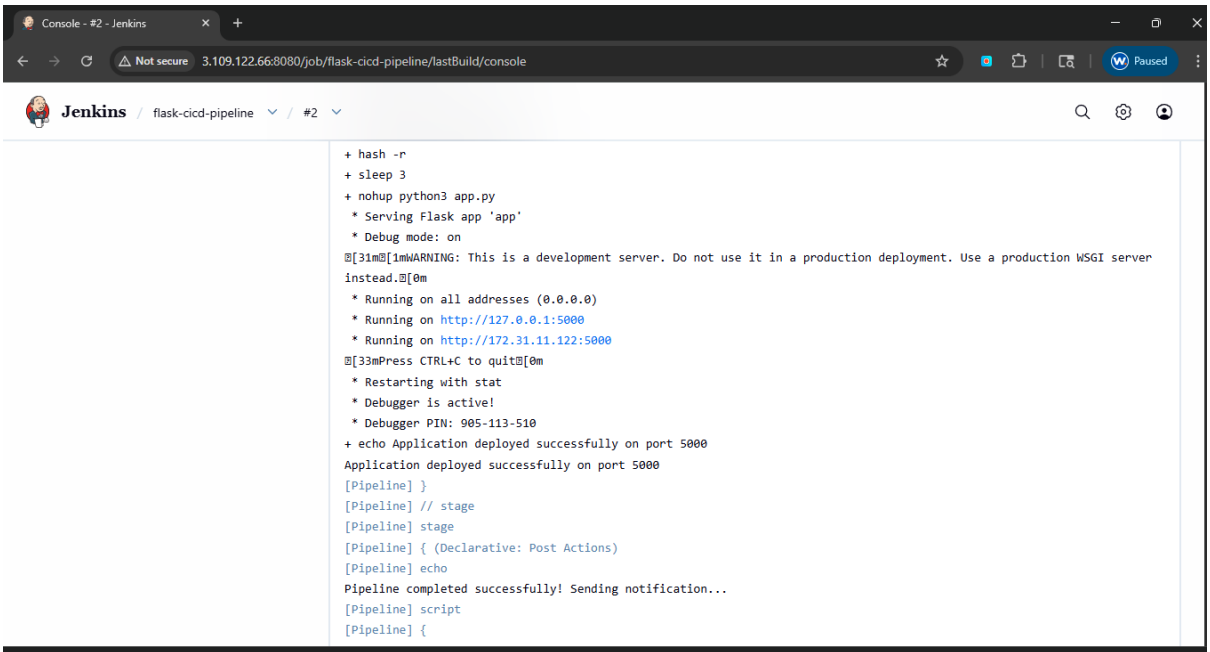
Console - #2 - Jenkins

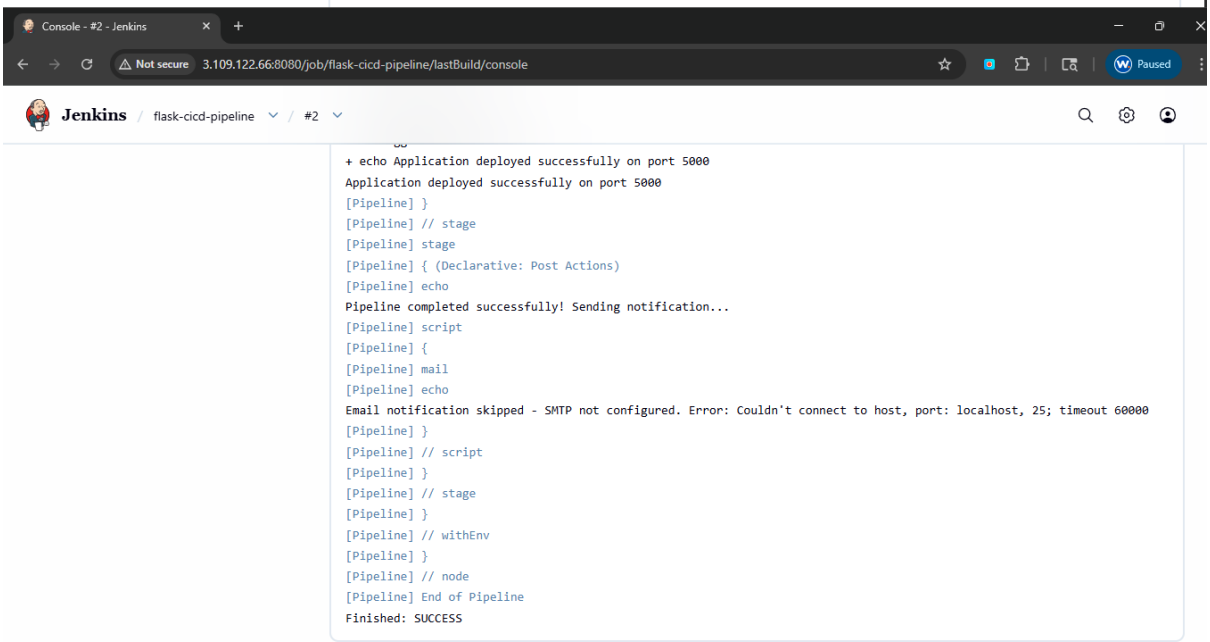
Not secure 3.109.122.66:8080/job/flask-cicd-pipeline/lastBuild/console

Jenkins / flask-cicd-pipeline / #2

```
+ . venv/bin/activate
+ deactivate nondestructive
+ [ -n  ]
+ [ -n  ]
+ hash -r
+ [ -n  ]
+ unset VIRTUAL_ENV
+ unset VIRTUAL_ENV_PROMPT
+ [ ! nondestructive = nondestructive ]
+ [  = cygwin ]
+ [  = msys ]
+ export VIRTUAL_ENV=/var/snap/jenkins/5013/workspace/flask-cicd-pipeline/venv
+ _OLD_VIRTUAL_PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
+ PATH=/var/snap/jenkins/5013/workspace/flask-cicd-
pipeline/venv/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
+ export PATH
+ [ -n  ]
+ [ -z  ]
+ _OLD_VIRTUAL_PS1=#
+ PS1=(venv) #
+ export PS1
+ VIRTUAL_ENV_PROMPT=(venv)
+ export VIRTUAL_ENV_PROMPT
+ hash -r
```

3.109.122.66:8080





This screenshot displays the Jenkins console output showing dependency installation, execution of pytest unit tests, deployment of the Flask application to the staging environment, and post-build notification handling.

2.4 Jenkins Triggers and Notifications

The pipeline is configured to run based on SCM execution from the main branch. Poll SCM and webhook triggers were not enabled as they are optional enhancements. Jenkins notifications are implemented using post-build actions in the pipeline. SMTP configuration is documented but not enabled as it is out of scope for this assignment.

3. GitHub Actions CI/CD Pipeline

3.1 Objective

To implement a CI/CD workflow using GitHub Actions that performs continuous integration, staging deployment, and production deployment for the Flask application.

3.2 Workflow Configuration

The GitHub Actions workflow is defined in the below file

`.github/workflows/flask-ci-cd.yml` file.

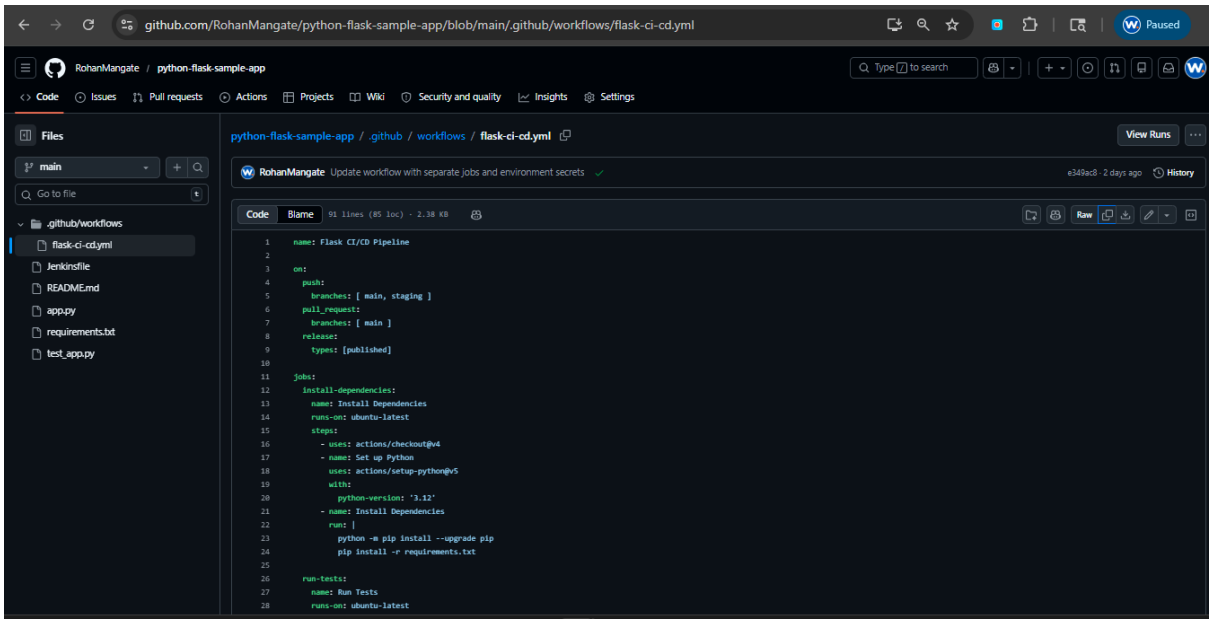
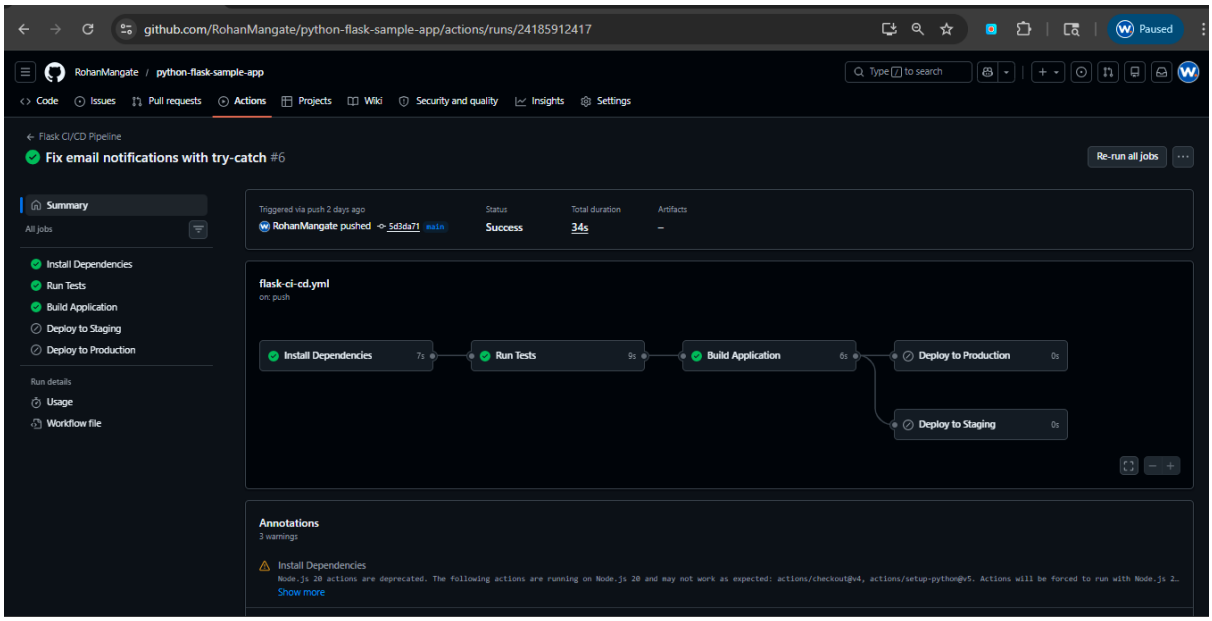


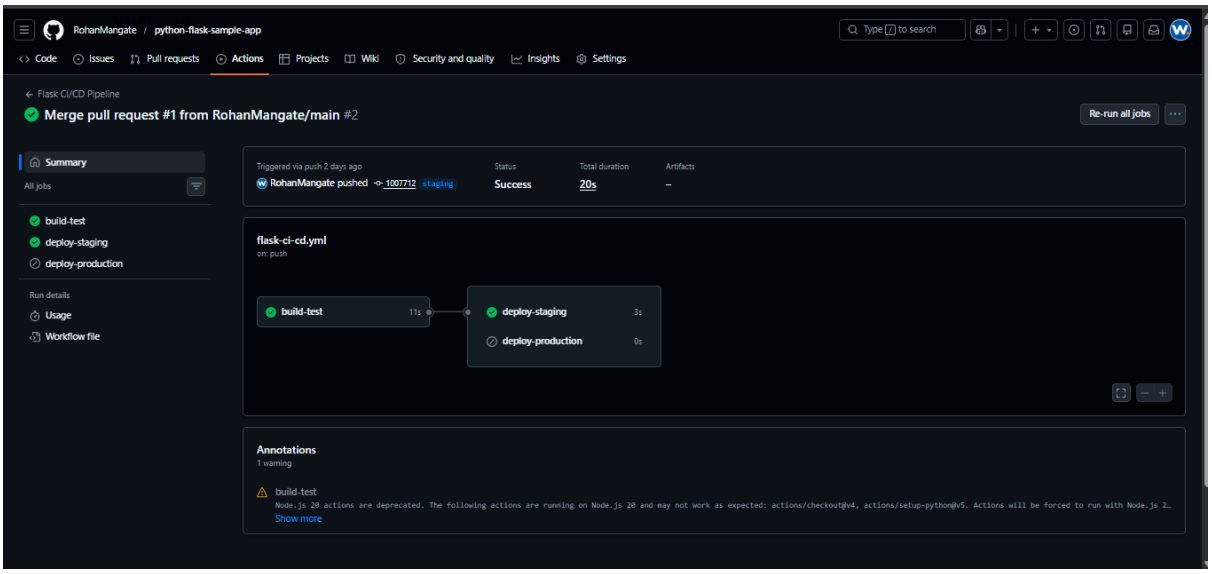
Figure 5: GitHub Actions CI/CD Workflow Definition



This screenshot shows the GitHub Actions workflow file defining jobs for installing dependencies, running tests, building the application, and deploying to staging and production environments.

3.3 Continuous Integration on Main Branch

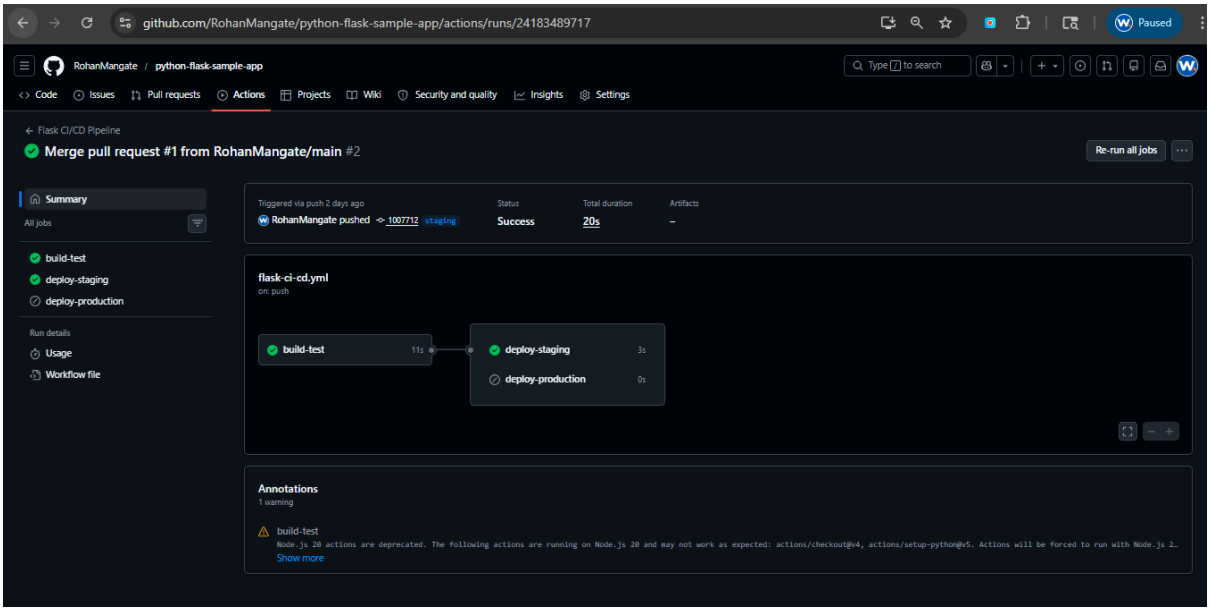
Figure 6: GitHub Actions CI Workflow Run on Main Branch



This screenshot shows a successful GitHub Actions workflow execution triggered by a push to the main branch. The workflow completed dependency installation, unit tests using pytest, and the build stage successfully.

3.4 Staging Deployment

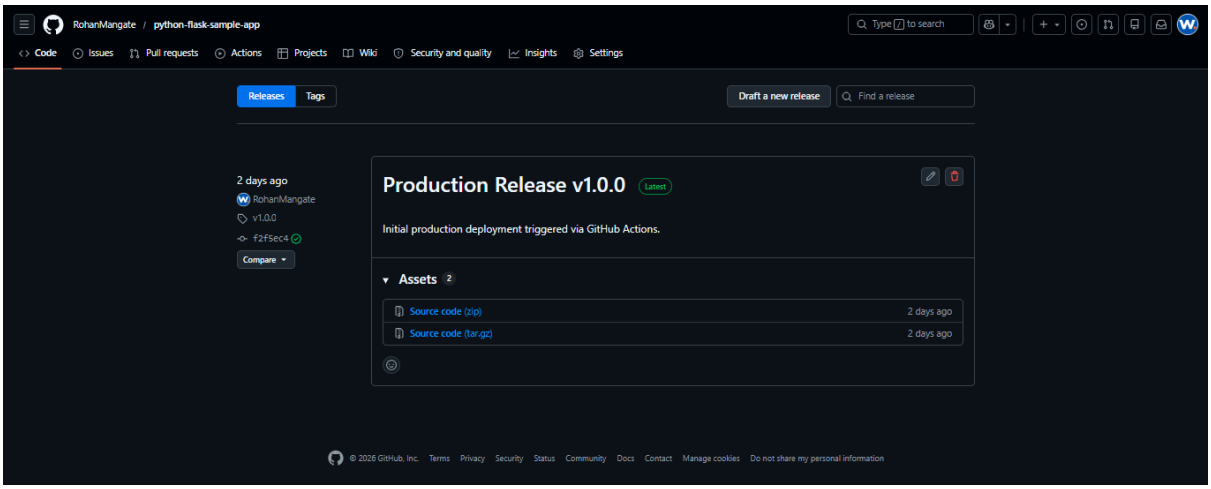
Figure 7: GitHub Actions Deployment to Staging Environment



This screenshot shows a successful GitHub Actions workflow run triggered by changes to the staging branch. The pipeline executed build and test jobs followed by successful deployment to the staging environment.

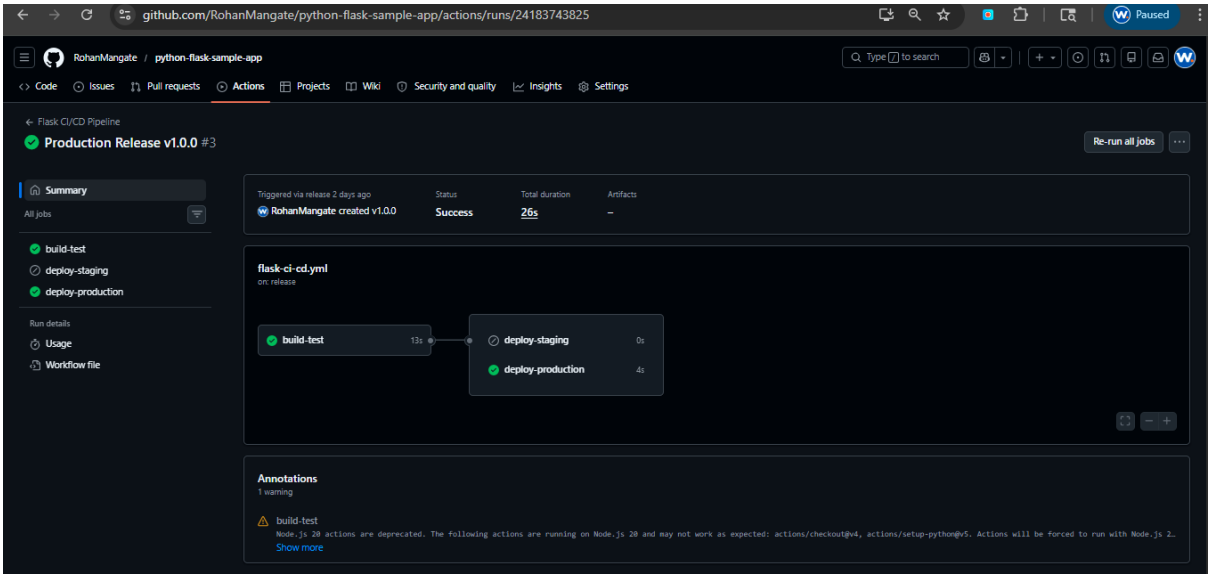
3.5 Production Deployment Using Release Tags

Figure 8: GitHub Production Release v1.0.0



This screenshot shows the GitHub production release v1.0.0 created for the application. The release tag is used to trigger the production deployment pipeline.

Figure 9: GitHub Actions Production Deployment Workflow Run



This screenshot shows a successful production deployment workflow run triggered by the v1.0.0 release. The pipeline completed build, test, and production deployment stages successfully.

4. Conclusion

This assignment successfully demonstrates end-to-end CI/CD implementation using Jenkins and GitHub Actions. Both pipelines automate build, test, and deployment processes, ensuring consistent and reliable delivery of the Flask application across staging and production environments.

5. Notes

- Jenkins SCM-based pipeline execution was used instead of Poll SCM.
- Jenkins email notifications are implemented logically and documented.
- GitHub Secrets are referenced for deployment security.
- All workflows executed successfully with visual proof.